

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Ingeniería · Ingeniería de Software

INFORME DE PRÁCTICA DE LABORATORIO*GA – Práctica en Clase | Semana 5 · Unidad II***Instalación de Entorno y Primer Script — PHP 8.2 y PERL/CGI**

Estudiante:	[Nombre completo del estudiante]
Código:	[Código estudiantil]
Asignatura:	Aplicaciones Web
Semana:	Semana 5 — Del 08 al 14 de Diciembre del 2025
Docente:	[Nombre del docente]
Fecha de entrega:	[DD/MM/AAAA]

Aplicaciones Web — SOFT-R-A — 5to Nivel | Corte 1 · UTEQ · 2025–2026

1. Introducción y Marco Teórico

La programación del lado del servidor constituye el fundamento sobre el cual se construyen las aplicaciones web dinámicas modernas. En este paradigma, el servidor procesa cada solicitud HTTP entrante, valida los datos proporcionados por el usuario, aplica las transformaciones de seguridad necesarias y genera una respuesta HTML que es enviada de regreso al navegador cliente. Dentro de este flujo, el procesamiento seguro de formularios representa uno de los puntos más críticos de la arquitectura, dado que los campos de entrada del usuario son el vector de ataque más explotado en aplicaciones web de acuerdo con la clasificación OWASP Top 10, donde la categoría A03:2021 —Injection— agrupa ataques que comprometen sistemas mediante datos maliciosos enviados a través de intérpretes [1]. PHP 8.2 y PERL 5.38 con el módulo CGI.pm representan dos enfoques arquitectónicamente distintos para resolver este problema: el primero opera como módulo nativo integrado al proceso de Apache, mientras el segundo sigue el paradigma del Common Gateway Interface (CGI) clásico, en el que el servidor lanza un proceso independiente del intérprete para cada solicitud recibida [2].

La mitigación de ataques de tipo Cross-Site Scripting (XSS) reflejado constituye el eje de seguridad central de esta práctica. Un ataque XSS reflejado ocurre cuando un script malicioso es introducido en un campo de formulario y, al ser devuelto sin tratamiento en la respuesta HTML del servidor, el navegador lo interpreta y ejecuta como código legítimo [3]. En PHP 8.2, la arquitectura de defensa se fundamenta en dos funciones complementarias: **filter_input()**, que lee los parámetros del método POST aplicando filtros de validación o saneamiento definidos por el motor interno del lenguaje, y **htmlspecialchars()**, que convierte los caracteres con significado especial en HTML —específicamente <, >, & y las comillas— en sus entidades HTML equivalentes, impidiendo que el navegador interprete el contenido como marcado ejecutable [4]. En PERL 5.38 con CGI.pm, el equivalente funcional de **htmlspecialchars()** es **CGI::escapeHTML()**, una función del módulo que aplica la misma transformación de entidades HTML sobre cualquier cadena de texto antes de su inclusión en la respuesta [5]. La validación del formato de correo electrónico ilustra otra diferencia arquitectónica relevante: mientras PHP proporciona el filtro nativo **FILTER_VALIDATE_EMAIL** basado en la implementación interna del estándar RFC 5321, PERL delega esta responsabilidad al desarrollador mediante el uso de expresiones regulares construidas manualmente, lo que otorga mayor flexibilidad pero requiere un conocimiento más profundo del lenguaje para garantizar la cobertura correcta de casos límite [6].

2. Configuración del Entorno de Desarrollo Local

El entorno de desarrollo empleado en esta práctica corresponde a una instalación local de XAMPP 8.2.x, una distribución de software libre que integra en un único paquete el servidor web Apache, el intérprete PHP 8.2, el servidor de bases de datos MariaDB y el intérprete PERL 5.38. Esta solución fue seleccionada por su amplia compatibilidad con sistemas operativos Windows, Linux y macOS, así como por la facilidad de configuración que ofrece a través de su panel de control gráfico, el cual permite iniciar y detener los servicios de Apache y MariaDB mediante una interfaz visual sin necesidad de intervención directa en la línea de comandos [2]. La instalación se realizó conservando las opciones predeterminadas del asistente de configuración, asegurando que los componentes Apache, PHP y Perl quedaran habilitados desde el inicio del proceso.

La estructura de directorios de XAMPP establece una separación funcional clara entre los distintos tipos de scripts del servidor. El directorio **htdocs/** —ubicado en `C:\xampp\htdocs\` en sistemas Windows o en `/opt/lampp/htdocs/` en entornos Linux y macOS— actúa como la raíz del servidor web y es el espacio designado para alojar los archivos HTML estáticos y los scripts PHP procesados directamente por el módulo **mod_php** integrado en Apache. Para los fines de esta práctica, se creó la subcarpeta **htdocs/practica5/**, dentro de la cual residen los archivos **formulario.html** y **procesar.php**. El directorio **cgi-bin/** —en `C:\xampp\cgi-bin\` para entornos Windows— constituye la ubicación reservada para los scripts PERL que operan bajo el protocolo CGI clásico. Apache identifica este directorio mediante la directiva **ScriptAlias** definida en su archivo de configuración **httpd.conf**, la cual instruye al servidor para que trate todos los archivos de ese directorio como programas ejecutables en lugar de contenido estático a servir. En esta carpeta se depositó el script **procesar.pl**, el cual requirió, en entornos Linux y macOS, la asignación de permisos de ejecución mediante el comando **chmod 755 procesar.pl** para que Apache pudiera invocarlo correctamente [2].

Figura 1. Entorno de Configuración del Servidor

[Marcador Analítico: Proyección del comportamiento de la interfaz en XAMPP — Servicios Apache habilitados para PHP y CGI para el Caso Entorno]

Figura 1. Proyección de la configuración del entorno local XAMPP con los servicios Apache habilitados para el procesamiento PHP y CGI.

3. Fase de Ejecución y Casos de Prueba

La presente sección documenta el análisis sistemático de los diez casos de prueba definidos en la guía de la práctica, estructurados para verificar el comportamiento arquitectónico de los scripts de procesamiento ante un espectro completo de entradas: desde datos completamente ausentes hasta vectores de inyección de código malicioso. Cada caso fue diseñado para ser evaluado de forma independiente contra ambos backends —PHP 8.2 y PERL 5.38/CGI— con el fin de contrastar directamente sus mecanismos de validación, saneamiento y generación de respuesta mediante un análisis estático de la lógica del código. Para cada caso se presenta el propósito técnico del escenario, la tabla de valores de entrada exactos definidos, el análisis del comportamiento arquitectónico esperado en cada tecnología, y los marcadores analíticos correspondientes a la proyección predictiva del comportamiento de la interfaz. Los casos de prueba 7 y 8, relativos a inyección XSS, incluyen adicionalmente un análisis técnico extendido del mecanismo de neutralización implementado por cada lenguaje [1][3].

Caso de Prueba 1 — Envío del formulario con todos los campos vacíos

Propósito Técnico

Este caso verifica que ambos scripts detecten y rechacen correctamente una solicitud HTTP POST en la que el usuario no proporcionó ningún valor en los campos del formulario. El objetivo es confirmar que las rutinas de validación de cada tecnología no permiten el procesamiento de datos ausentes y que los mensajes de error se generan de forma individual e independiente para cada campo obligatorio, sin que la ausencia de un campo afecte la evaluación de los demás [4][5].

Valores de Entrada

Tabla 2. Valores de entrada para el Caso de Prueba 1 — Formulario enviado sin completar ningún campo.

Campo de entrada	Valor ingresado
nombre	(vacío — sin valor)
email	(vacío — sin valor)
edad	(vacío — sin valor)
mensaje	(vacío — sin valor)

Comportamiento Arquitectónico Esperado en PHP 8.2

PHP 8.2 — Evaluación Estática de la Lógica del Código

La función `filter_input(INPUT_POST, 'nombre', FILTER_DEFAULT)` recupera una cadena vacía del método POST. Tras aplicar `trim()` y `htmlspecialchars()`, la función `empty()` evalúa el campo como vacío e inserta el mensaje de error correspondiente en el arreglo `$errores`. El mismo proceso se repite de manera análoga para el campo mensaje. En el caso del correo electrónico, el filtro `FILTER_VALIDATE_EMAIL` devuelve false ante una cadena vacía, ya que esta no satisface ningún patrón de formato de dirección de correo. La función `FILTER_VALIDATE_INT` con las opciones de rango opera de forma equivalente para el campo edad, retornando igualmente false. Al concluir las cuatro validaciones, el arreglo `$errores` contiene exactamente cuatro entradas, una por cada campo, y la

plantilla HTML resultante renderizará los cuatro mensajes de error con el formato visual diferenciado definido en la clase CSS `.error` [4].

Comportamiento Arquitectónico Esperado en PERL 5.38/CGI

PERL 5.38/CGI — Evaluación Estática de la Lógica del Código

El método `$cgi->param('nombre')` retorna una cadena vacía del objeto CGI. La función `sanear()` aplica `CGI::escapeHTML()` y elimina espacios mediante la expresión regular `s/^\s+|\s+$//g`, resultando igualmente en cadena vacía. La condición `length($nombre) < 2` se evalúa como verdadera y el error de nombre queda registrado en el hash `%errores`. Para el correo electrónico, la expresión regular de validación `/^[a-zA-Z0-9._%+\-]+\@[a-zA-Z0-9.\-]+\.[a-zA-Z]{2,}$/` no puede coincidir con una cadena vacía, por lo que la cláusula `unless` activa el registro del error. Para la edad, el patrón `/^\d+$/` tampoco coincide con una cadena vacía, por lo que se registra el error correspondiente. Al finalizar las validaciones, el hash `%errores` acumula los cuatro mensajes de error, que son impresos secuencialmente en el bloque HTML de respuesta mediante el bucle `foreach` [5].

[Marcador Analítico: Proyección del comportamiento de la interfaz en PHP 8.2 para el Caso 1]

Figura 2. Proyección del comportamiento de la interfaz de PHP 8.2 ante el Caso de Prueba 1.

[Marcador Analítico: Proyección del comportamiento de la interfaz en PERL 5.38/CGI para el Caso 1]

Figura 3. Proyección del comportamiento de la interfaz de PERL 5.38/CGI ante el Caso de Prueba 1.

Caso de Prueba 2 — Correo electrónico sin dominio posterior al símbolo arroba

Propósito Técnico

Este caso verifica que la validación del campo de correo electrónico detecte específicamente un formato incompleto en el que el símbolo `@` está presente pero la parte correspondiente al dominio se encuentra ausente. Los demás campos se completan con valores válidos con el propósito de aislar el error exclusivamente al campo email, comprobando que el sistema no genera falsos positivos en los campos restantes [4][5].

Valores de Entrada

Tabla 3. Valores de entrada para el Caso de Prueba 2 — Dirección de correo sin dominio.

Campo de entrada	Valor ingresado
nombre	Ana
email	usuario@
edad	20
mensaje	Hola

Comportamiento Arquitectónico Esperado en PHP 8.2

PHP 8.2 — Evaluación Estática de la Lógica del Código

Los campos nombre, edad y mensaje superan sus validaciones respectivas sin inconvenientes, dado que todos contienen valores que cumplen los criterios de tipo, longitud y rango definidos. La función `filter_input(INPUT_POST, 'email', FILTER_VALIDATE_EMAIL)` aplica internamente una verificación del estándar RFC 5321. La cadena "usuario@" no satisface el patrón requerido puesto que carece completamente de la parte del dominio posterior al símbolo @, por lo que el filtro devuelve false. En consecuencia, únicamente el campo email es registrado en el arreglo \$errores, y la respuesta HTML presentará de forma exclusiva el mensaje de formato de correo inválido [4].

Comportamiento Arquitectónico Esperado en PERL 5.38/CGI

PERL 5.38/CGI — Evaluación Estática de la Lógica del Código

La expresión regular de validación `/^[a-zA-Z0-9._%+\-]+\@[a-zA-Z0-9.\-]+\.[a-zA-Z]{2,}$/` requiere que tras el símbolo @ exista al menos un carácter de dominio válido seguido de un punto y una extensión de dos o más letras. La cadena "usuario@" no cumple esta condición estructural, por lo que la cláusula `unless` agrega el error de email al hash %errores. Los campos nombre, edad y mensaje pasan sus validaciones con normalidad, de modo que la respuesta del servidor imprimirá únicamente el mensaje de formato de correo electrónico inválido [5].

[Marcador Analítico: Proyección del comportamiento de la interfaz en PHP 8.2 para el Caso 2]

Figura 4. Proyección del comportamiento de la interfaz de PHP 8.2 ante el Caso de Prueba 2.

[Marcador Analítico: Proyección del comportamiento de la interfaz en PERL 5.38/CGI para el Caso 2]

Figura 5. Proyección del comportamiento de la interfaz de PERL 5.38/CGI ante el Caso de Prueba 2.

Caso de Prueba 3 — Correo electrónico sin el símbolo arroba

Propósito Técnico

Este caso confirma que la validación del campo de correo electrónico detecta la ausencia del símbolo @, el cual constituye un elemento estructural indispensable en toda dirección de correo válida conforme al estándar RFC 5321. El resto de los campos se proporcionan con valores correctos para aislar el error al campo email y verificar el correcto funcionamiento de la validación ante una cadena que, aunque formalmente parece una dirección de correo, carece del separador obligatorio entre la parte local y el dominio [4][5].

Valores de Entrada

Tabla 4. Valores de entrada para el Caso de Prueba 3 — Cadena de correo electrónico sin símbolo arroba.

Campo de entrada	Valor ingresado
nombre	Carlos
email	usuariodominio.com
edad	25
mensaje	Prueba sin arroba

Comportamiento Arquitectónico Esperado en PHP 8.2

PHP 8.2 — Evaluación Estática de la Lógica del Código

El filtro `FILTER_VALIDATE_EMAIL` evalúa la cadena "usuariodominio.com" y determina que no corresponde a un formato de dirección de correo válido, al no encontrar el carácter separador `@` que divide la parte local del dominio. La función retorna false, lo que provoca que únicamente el campo email sea registrado en el arreglo \$errores. Los campos nombre, edad y mensaje, procesados con sus respectivos mecanismos de validación, no generan error alguno, y la respuesta HTML presentará exclusivamente el mensaje de formato inválido para el correo electrónico [4].

Comportamiento Arquitectónico Esperado en PERL 5.38/CGI**PERL 5.38/CGI — Evaluación Estática de la Lógica del Código**

La expresión regular de validación busca explícitamente el carácter `\@` como separador obligatorio entre la parte local y el dominio. Al estar ausente en la cadena "usuariodominio.com", el operador `unless` detecta que la condición de coincidencia no se satisface y registra el error en el hash %errores. Los demás campos superan sus validaciones con normalidad, por lo que el hash contendrá únicamente la entrada correspondiente al email y la respuesta HTML imprimirá un único mensaje de error [5].

[Marcador Analítico: Proyección del comportamiento de la interfaz en PHP 8.2 para el Caso 3]

Figura 6. Proyección del comportamiento de la interfaz de PHP 8.2 ante el Caso de Prueba 3.

[Marcador Analítico: Proyección del comportamiento de la interfaz en PERL 5.38/CGI para el Caso 3]

Figura 7. Proyección del comportamiento de la interfaz de PERL 5.38/CGI ante el Caso de Prueba 3.

Caso de Prueba 4 — Edad con valor numérico negativo**Propósito Técnico**

Este caso comprueba que la validación de rango rechace valores numéricos enteros que se encuentran por debajo del límite inferior permitido, establecido en 1. Un valor negativo como -5, si bien es un entero en sentido matemático, no representa una edad válida en el contexto del formulario. Los demás campos se ingresan con valores correctos para verificar que el error queda aislado exclusivamente al campo edad, sin interferir en la evaluación de los campos restantes [4][5].

Valores de Entrada

Tabla 5. Valores de entrada para el Caso de Prueba 4 — Edad con valor negativo fuera del rango permitido.

Campo de entrada	Valor ingresado
nombre	Laura
email	laura@correo.com
edad	-5
mensaje	Prueba edad negativa

Comportamiento Arquitectónico Esperado en PHP 8.2

PHP 8.2 — Evaluación Estática de la Lógica del Código

La función `filter_input(INPUT_POST, 'edad', FILTER_VALIDATE_INT, $opciones_edad)` aplica la restricción `'min_range' => 1` definida en el arreglo de opciones. El valor `-5` es reconocido por el motor de PHP como un número entero en formato, pero al ser inferior al mínimo permitido, la función retorna `false`. Esta devolución activa el registro del error de rango en el arreglo `$errores` con el mensaje correspondiente. Los campos `nombre`, `email` y `mensaje` son procesados y validados sin incidencias, de modo que el arreglo acumulará únicamente un error [4].

Comportamiento Arquitectónico Esperado en PERL 5.38/CGI**PERL 5.38/CGI — Evaluación Estática de la Lógica del Código**

La expresión regular `/^\d+$/` verifica que la cadena ingresada esté compuesta exclusivamente por dígitos decimales sin ningún otro carácter. La cadena `"-5"` contiene el símbolo negativo `-`, que no pertenece al conjunto de dígitos `\d`, por lo que la expresión regular no coincide y la condición `unless` se activa inmediatamente sin necesidad de evaluar el rango numérico posterior. El hash `%errores` contendrá únicamente la entrada correspondiente a la edad, y la respuesta imprimirá un solo mensaje de error. Esta diferencia respecto a PHP 8.2 es técnicamente significativa: PERL rechaza el valor negativo por falla en la validación de tipo (el signo `-`), mientras que PHP lo rechaza por falla en la validación de rango (inferior a 1) [5][6].

[Marcador Analítico: Proyección del comportamiento de la interfaz en PHP 8.2 para el Caso 4]

Figura 8. Proyección del comportamiento de la interfaz de PHP 8.2 ante el Caso de Prueba 4.

[Marcador Analítico: Proyección del comportamiento de la interfaz en PERL 5.38/CGI para el Caso 4]

Figura 9. Proyección del comportamiento de la interfaz de PERL 5.38/CGI ante el Caso de Prueba 4.

Caso de Prueba 5 — Edad con valor superior al máximo permitido**Propósito Técnico**

Este caso verifica la restricción del límite superior del rango para el campo `edad`. A diferencia del Caso de Prueba 4, el valor `200` es un entero positivo que supera la validación de tipo numérico pero excede el máximo permitido de `120`. El propósito es confirmar que ambas tecnologías aplican correctamente la validación de rango en su límite superior y que los demás campos no se ven afectados por el error aislado en el campo `edad` [4][5].

Valores de Entrada

Tabla 6. Valores de entrada para el Caso de Prueba 5 — Edad con valor entero positivo superior al máximo de 120.

Campo de entrada	Valor ingresado
nombre	Pedro
email	pedro@correo.com
edad	200

Campo de entrada	Valor ingresado
mensaje	Prueba edad maxima

Comportamiento Arquitectónico Esperado en PHP 8.2

PHP 8.2 — Evaluación Estática de la Lógica del Código

El filtro `FILTER_VALIDATE_INT` con la opción `'max_range' => 120` evalúa el valor 200. Aunque la cadena es reconocida como un entero válido en formato, supera el límite máximo configurado y la función retorna false. El campo edad queda registrado en el arreglo `$errores` con el mensaje de fuera de rango. Los campos nombre, email y mensaje son procesados sin incidencias. Esta implementación de PHP 8.2 mediante `FILTER_VALIDATE_INT` resulta más concisa que una verificación manual, ya que delega la lógica de rango al motor interno del lenguaje en lugar de requerir comparaciones explícitas adicionales en el código del desarrollador [4].

Comportamiento Arquitectónico Esperado en PERL 5.38/CGI

PERL 5.38/CGI — Evaluación Estática de la Lógica del Código

La expresión regular `/^\d+$/` confirma en primera instancia que la cadena "200" está compuesta exclusivamente por dígitos decimales, por lo que la validación de tipo es superada. Sin embargo, la evaluación posterior de la condición compuesta `$edad >= 1 && $edad <= 120` resulta falsa, ya que 200 excede el valor máximo de 120. La conjunción lógica `&&` dentro de la cláusula `unless` hace que todo el bloque sea evaluado como verdadero y el error sea registrado en el hash `%errores`. A diferencia del Caso de Prueba 4, en este escenario PERL sí evalúa el rango numérico porque el valor pasa primero la verificación de tipo mediante la expresión regular. Esto ilustra la naturaleza secuencial de la validación en PERL, donde el tipo y el rango se verifican en pasos explícitamente diferenciados por el desarrollador, en contraste con la validación unificada de PHP mediante `FILTER_VALIDATE_INT` [5][6].

[Marcador Analítico: Proyección del comportamiento de la interfaz en PHP 8.2 para el Caso 5]

Figura 10. Proyección del comportamiento de la interfaz de PHP 8.2 ante el Caso de Prueba 5.

[Marcador Analítico: Proyección del comportamiento de la interfaz en PERL 5.38/CGI para el Caso 5]

Figura 11. Proyección del comportamiento de la interfaz de PERL 5.38/CGI ante el Caso de Prueba 5.

Caso de Prueba 6 — Edad ingresada como cadena de texto no numérica

Propósito Técnico

Este caso verifica el comportamiento de los scripts cuando el campo edad recibe una cadena de texto alfabético en lugar de un valor numérico entero. A diferencia de los casos anteriores, en este escenario el valor no constituye un número fuera de rango sino un tipo de dato estructuralmente incorrecto, lo que permite evaluar la robustez de los mecanismos de validación de tipo de cada tecnología ante entradas malformadas o errores involuntarios del usuario [4][5].

Valores de Entrada

Tabla 7. Valores de entrada para el Caso de Prueba 6 — Edad expresada como palabra en lugar de dígitos.

Campo de entrada	Valor ingresado
nombre	Sofía
email	sofia@correo.com
edad	veinte
mensaje	Prueba edad en texto

Comportamiento Arquitectónico Esperado en PHP 8.2

PHP 8.2 — Evaluación Estática de la Lógica del Código

El filtro `FILTER_VALIDATE_INT` intenta interpretar la cadena "veinte" como un número entero. Al tratarse de una secuencia de caracteres alfabéticos sin ningún dígito, la conversión no es posible y la función retorna `false` de inmediato. Esta constituye una ventaja característica del uso de filtros nativos en PHP 8.2: no se requiere lógica adicional para distinguir entre un error de tipo y un error de rango, dado que ambos escenarios producen `false` y activan el mismo mensaje de error hacia el usuario. El campo edad queda registrado en el arreglo `$errores` mientras que los campos nombre, email y mensaje superan sus validaciones respectivas sin incidencias [4].

Comportamiento Arquitectónico Esperado en PERL 5.38/CGI

PERL 5.38/CGI — Evaluación Estática de la Lógica del Código

La expresión regular `/^\d+$/` verifica que la cadena ingresada contenga únicamente dígitos decimales del cero al nueve. La cadena "veinte" está compuesta exclusivamente por letras minúsculas, por lo que la expresión regular no produce coincidencia alguna. La cláusula `unless` se activa de manera inmediata y registra el error en el hash `%errores` antes de llegar a evaluar la condición de rango numérico, ya que la conjunción lógica `&&` provoca un cortocircuito de evaluación en cuanto el primer operando resulta falso. Este comportamiento es técnicamente consistente con el observado en el Caso de Prueba 4, donde el valor negativo también era rechazado en la fase de verificación de tipo antes de alcanzar la verificación de rango, lo que confirma la naturaleza secuencial y explícita de la validación en PERL [5][6].

[Marcador Analítico: Proyección del comportamiento de la interfaz en PHP 8.2 para el Caso 6]

Figura 12. Proyección del comportamiento de la interfaz de PHP 8.2 ante el Caso de Prueba 6.

[Marcador Analítico: Proyección del comportamiento de la interfaz en PERL 5.38/CGI para el Caso 6]

Figura 13. Proyección del comportamiento de la interfaz de PERL 5.38/CGI ante el Caso de Prueba 6.

Caso de Prueba 7 — Intento de inyección XSS mediante etiqueta script en el campo nombre

Propósito Técnico

Este caso evalúa la resistencia arquitectónica de ambos scripts ante un ataque de tipo Cross-Site Scripting (XSS) reflejado en su forma más directa. El vector de ataque consiste en introducir una etiqueta `<script>` completa con una función `alert()` en el campo nombre, con el objetivo de que el navegador cliente interprete y ejecute el código JavaScript inyectado en el momento en que se renderiza la respuesta del servidor. Si el mecanismo de saneamiento no estuviera implementado, el script inyectado se ejecutaría produciendo una alerta visible en el navegador, lo que en aplicaciones web reales representaría una vulnerabilidad crítica capaz de comprometer sesiones de usuario, redirigir tráfico o exfiltrar datos sensibles almacenados en el contexto del navegador [1][3].

Valores de Entrada

Tabla 8. Valores de entrada para el Caso de Prueba 7 — Inyección XSS mediante etiqueta script en el campo nombre.

Campo de entrada	Valor ingresado
nombre	<code><script>alert(1)</script></code>
email	<code>xss@prueba.com</code>
edad	<code>22</code>
mensaje	<code>Prueba XSS en nombre</code>

Comportamiento Arquitectónico Esperado en PHP 8.2

PHP 8.2 — Evaluación Estática de la Lógica del Código

El valor del campo nombre es leído por `filter_input(INPUT_POST, 'nombre', FILTER_DEFAULT)`, que en este contexto recupera el valor sin aplicar ningún filtro de transformación de tipo. A continuación, la función `htmlspecialchars($nombre_raw, ENT_QUOTES, 'UTF-8')` procesa la cadena carácter a carácter y convierte cada símbolo con significado especial en HTML en su entidad de carácter correspondiente: el símbolo `<` se transforma en `<`, el símbolo `>` en `>`, y las comillas dobles y simples en `"` y `'` respectivamente, al estar habilitada la constante `ENT_QUOTES`. La cadena que se insertará en el documento HTML de respuesta es `<script>alert(1)</script>`, que el navegador renderizará como texto plano visible en pantalla y no como código JavaScript ejecutable. El análisis estático del código fuente HTML generado confirma que los delimitadores de la etiqueta habrán sido reemplazados por sus entidades, lo que impide su interpretación como marcado [4].

Comportamiento Arquitectónico Esperado en PERL 5.38/CGI

PERL 5.38/CGI — Evaluación Estática de la Lógica del Código

La función `sanear()` invoca `CGI::escapeHTML($texto)`, que aplica la misma transformación de entidades HTML sobre cualquier cadena recibida. La cadena `"<script>alert(1)</script>"` quedará convertida en `"<script>alert(1)</script>"` antes de ser almacenada en la variable `$nombre`. Cuando esta variable es impresa mediante `print` dentro del bloque HTML de respuesta, el navegador recibirá una cadena de texto que describe visualmente la etiqueta pero que carece de los delimitadores funcionales `<` y `>`, neutralizando por completo el vector de ataque. El análisis estático del código fuente

HTML generado confirma que el contenido del campo nombre aparecerá codificado como entidades y no como etiquetas ejecutables [5].

Análisis Técnico de Saneamiento contra XSS

⚠ Análisis de Seguridad — Mecanismo de Neutralización XSS

El mecanismo de defensa contra XSS reflejado se fundamenta en la transformación sistemática de los delimitadores de etiquetas HTML (< y >) en entidades de carácter antes de escribirlos en el documento de respuesta. Cuando el navegador recibe la entidad < la renderiza como el carácter literal < en pantalla, pero nunca la interpreta como el inicio de una etiqueta HTML ejecutable, dado que el motor de renderizado del navegador distingue entre entidades de carácter y marcado estructural. De este modo, el código malicioso inyectado es degradado a datos de texto inofensivos. Tanto `htmlspecialchars()` en PHP 8.2 como `CGI::escapeHTML()` en PERL 5.38 son funcionalmente equivalentes en este objetivo de neutralización, y su aplicación sobre toda salida dinámica generada a partir de entradas del usuario constituye la medida de mitigación primaria contra XSS reflejado definida por OWASP [1][3][7].

[Marcador Analítico: Proyección del comportamiento de la interfaz en PHP 8.2 para el Caso 7]

Figura 14. Proyección del comportamiento de la interfaz para el Caso de Prueba 7 — PHP 8.2.

[Marcador Analítico: Proyección del comportamiento de la interfaz en PHP 8.2 — Código Fuente Generado para el Caso 7]

Figura 15. Proyección del comportamiento de la interfaz para el Caso de Prueba 7 — PHP 8.2 — Código Fuente Generado.

[Marcador Analítico: Proyección del comportamiento de la interfaz en PERL 5.38/CGI para el Caso 7]

Figura 16. Proyección del comportamiento de la interfaz para el Caso de Prueba 7 — PERL 5.38/CGI.

[Marcador Analítico: Proyección del comportamiento de la interfaz en PERL 5.38/CGI — Código Fuente Generado para el Caso 7]

Figura 17. Proyección del comportamiento de la interfaz para el Caso de Prueba 7 — PERL 5.38/CGI — Código Fuente Generado.

Caso de Prueba 8 — Intento de inyección XSS mediante atributo de evento en el campo mensaje

Propósito Técnico

Este caso evalúa una variante técnicamente más sofisticada del ataque XSS reflejado, en la que el vector de ataque no utiliza una etiqueta <script> directa sino una etiqueta HTML aparentemente legítima —la etiqueta — combinada con el atributo de evento del DOM `onerror` para ejecutar código JavaScript de forma indirecta. Esta técnica es capaz de eludir filtros de seguridad rudimentarios que únicamente buscan y bloquean la cadena <script>, ya que el código malicioso se inyecta a través de un manejador de evento válido en la especificación HTML. La prueba busca verificar que ambos scripts neutralizan este vector con la misma efectividad que el caso anterior, dado que la protección no discrimina entre tipos de etiquetas [1][3].

Valores de Entrada

Tabla 9. Valores de entrada para el Caso de Prueba 8 — Inyección XSS mediante etiqueta img con atributo de evento onerror en el campo mensaje.

Campo de entrada	Valor ingresado
nombre	Miguel
email	miguel@correo.com
edad	30
mensaje	

Comportamiento Arquitectónico Esperado en PHP 8.2**PHP 8.2 — Evaluación Estática de la Lógica del Código**

La cadena "" es procesada por `htmlspecialchars($mensaje_raw, ENT_QUOTES, 'UTF-8')`. La función transforma el símbolo < inicial de la etiqueta en < y el símbolo > de cierre en >. La cadena resultante es insertada en el documento HTML de respuesta como texto plano. El navegador no puede interpretar esta cadena como una etiqueta funcional porque los delimitadores que le confieren significado estructural han sido reemplazados por sus entidades de carácter. En consecuencia, el atributo onerror nunca llega a ser registrado por el motor de eventos del DOM ni puede ser ejecutado, dado que el elemento imagen nunca llega a existir como nodo del árbol de objetos del documento. El análisis estático del código fuente HTML generado confirma la presencia de las entidades en lugar de los delimitadores originales [4].

Comportamiento Arquitectónico Esperado en PERL 5.38/CGI**PERL 5.38/CGI — Evaluación Estática de la Lógica del Código**

La función `CGI::escapeHTML()` dentro de `sanear()` aplica la misma transformación de entidades sobre la cadena de entrada. El atributo onerror y su valor quedan encapsulados dentro de la representación textual de la etiqueta , que el navegador interpretará exclusivamente como una secuencia de caracteres de texto y no como marcado HTML estructural. Este comportamiento confirma que `CGI::escapeHTML()` es funcionalmente equivalente a `htmlspecialchars()` de PHP para la prevención de ataques XSS reflejados basados en atributos de evento, independientemente de la complejidad del vector empleado. El análisis estático del código fuente HTML generado confirmará que los delimitadores de la etiqueta aparecen codificados como entidades y que el atributo onerror no podrá ser interpretado por el navegador [5].

Análisis Técnico de Saneamiento contra XSS**⚠ Análisis de Seguridad — Mecanismo de Neutralización XSS**

La efectividad de la protección ante esta variante de XSS se debe a que `htmlspecialchars()` y `CGI::escapeHTML()` no operan sobre la semántica de las etiquetas HTML sino sobre los caracteres delimitadores que las constituyen. Cualquier secuencia que comience con < y termine con > recibe el

mismo tratamiento de codificación sin excepción, independientemente del nombre de la etiqueta o de los atributos que contenga. Esto convierte a la codificación de salida en una defensa genérica y exhaustiva contra toda variante de inyección HTML directa, incluyendo las técnicas que aprovechan manejadores de evento como `onerror`, `onload` u `onclick`, sin requerir un listado de etiquetas o atributos prohibidos que deba mantenerse actualizado. Esta propiedad de universalidad es precisamente la razón por la que OWASP recomienda la codificación de salida contextual como la medida de mitigación principal contra XSS, por encima de los enfoques basados en listas negras [1][3][7].

[Marcador Analítico: Proyección del comportamiento de la interfaz en PHP 8.2 para el Caso 8]

Figura 18. Proyección del comportamiento de la interfaz para el Caso de Prueba 8 — PHP 8.2.

[Marcador Analítico: Proyección del comportamiento de la interfaz en PHP 8.2 — Código Fuente Generado para el Caso 8]

Figura 19. Proyección del comportamiento de la interfaz para el Caso de Prueba 8 — PHP 8.2 — Código Fuente Generado.

[Marcador Analítico: Proyección del comportamiento de la interfaz en PERL 5.38/CGI para el Caso 8]

Figura 20. Proyección del comportamiento de la interfaz para el Caso de Prueba 8 — PERL 5.38/CGI.

[Marcador Analítico: Proyección del comportamiento de la interfaz en PERL 5.38/CGI — Código Fuente Generado para el Caso 8]

Figura 21. Proyección del comportamiento de la interfaz para el Caso de Prueba 8 — PERL 5.38/CGI — Código Fuente Generado.

Caso de Prueba 9 — Envío con todos los campos correctamente válidos

Propósito Técnico

Este caso constituye el escenario de flujo positivo de la práctica, conocido en ingeniería de software como *happy path*. Su objetivo es verificar que, cuando todos los campos del formulario contienen datos que satisfacen íntegramente los criterios de validación establecidos, ambos scripts procesan la información sin generar error alguno y presentan al usuario un resumen tabular con los valores recibidos y saneados. Este caso es igualmente relevante que los escenarios de error, dado que confirma que los mecanismos de validación no rechazan entradas legítimas ni producen falsos negativos [4][5].

Valores de Entrada

Tabla 10. Valores de entrada para el Caso de Prueba 9 — Todos los campos con datos correctos dentro de los rangos y formatos permitidos.

Campo de entrada	Valor ingresado
nombre	Juan Pablo Ramirez
email	juanpablo@correo.com
edad	22
mensaje	Este es un mensaje de prueba valido con datos correctos.

Comportamiento Arquitectónico Esperado en PHP 8.2

PHP 8.2 — Evaluación Estática de la Lógica del Código

Cada campo supera su validación correspondiente de forma secuencial. El nombre, tras aplicar trim() y htmlspecialchars(), contiene entre 2 y 100 caracteres y no está vacío. El filtro FILTER_VALIDATE_EMAIL confirma que la cadena "juanpablo@correo.com" satisface el formato de dirección de correo electrónico conforme al estándar RFC 5321. El filtro FILTER_VALIDATE_INT con opciones de rango acepta el valor 22 como entero dentro del intervalo 1–120. El mensaje, tras saneamiento y verificación de longitud, no está vacío ni supera los 500 caracteres. Al concluir las cuatro validaciones, el arreglo \$errores permanece vacío, por lo que la condición empty(\$errores) es verdadera y la plantilla HTML entra en la rama else, renderizando la tabla de resumen con los cuatro valores recibidos y el mensaje de confirmación de éxito [4].

Comportamiento Arquitectónico Esperado en PERL 5.38/CGI

PERL 5.38/CGI — Evaluación Estática de la Lógica del Código

Ninguna de las condiciones de error se activa durante las cuatro validaciones. La longitud del nombre supera el mínimo de 2 caracteres y no excede el máximo de 100. La expresión regular de email produce una coincidencia completa con "juanpablo@correo.com". La expresión regular /^d+\$/ confirma que "22" es una cadena de dígitos y la evaluación de rango confirma que el valor está dentro del intervalo 1–120. La longitud del mensaje es mayor a cero y no supera los 500 caracteres. El hash %errores queda vacío al finalizar las validaciones, por lo que la evaluación booleana if (%errores) resulta falsa y el bloque else imprimirá la tabla HTML con los cuatro valores saneados junto con el mensaje de confirmación de datos recibidos y validados correctamente [5].

[Marcador Analítico: Proyección del comportamiento de la interfaz en PHP 8.2 para el Caso 9]

Figura 22. Proyección del comportamiento de la interfaz de PHP 8.2 ante el Caso de Prueba 9.

[Marcador Analítico: Proyección del comportamiento de la interfaz en PERL 5.38/CGI para el Caso 9]

Figura 23. Proyección del comportamiento de la interfaz de PERL 5.38/CGI ante el Caso de Prueba 9.

Caso de Prueba 10 — Nombre con caracteres especiales del idioma español

Propósito Técnico

Este caso verifica que el campo nombre acepta correctamente cadenas que contienen caracteres propios del idioma español, tales como vocales con tilde, la letra ñ y el guion medio. Estos caracteres son plenamente válidos en nombres de personas de habla hispana y no deben ser rechazados ni alterados por los mecanismos de validación de longitud ni por el proceso de saneamiento. El objetivo es confirmar que la codificación UTF-8 declarada en ambos scripts permite representar y retransmitir estos caracteres con fidelidad tipográfica completa en la respuesta HTML [4][5][7].

Valores de Entrada

Tabla 11. Valores de entrada para el Caso de Prueba 10 — Nombre compuesto con tildes, letra ñ y guion medio.

Campo de entrada	Valor ingresado
nombre	Maria Jose Muñoz-Lopez
email	mjose@correo.com

Campo de entrada	Valor ingresado
edad	28
mensaje	Nombre con caracteres especiales del español.

Comportamiento Arquitectónico Esperado en PHP 8.2

PHP 8.2 — Evaluación Estática de la Lógica del Código

La función `htmlspecialchars($nombre_raw, ENT_QUOTES, 'UTF-8')` procesa únicamente los caracteres que poseen significado especial en el contexto del marcado HTML: los símbolos `<`, `>`, `&` y las comillas. Los caracteres con tilde, la letra ñ y el guion medio no pertenecen a este conjunto, por lo que son transmitidos sin modificación desde la cadena de entrada hasta la cadena de salida. La validación de longitud mediante `strlen()` opera correctamente sobre la cadena UTF-8; dado que los nombres con caracteres del español se mantienen ampliamente por debajo del límite de 100 caracteres, esta distinción entre bytes y caracteres no afecta el resultado de la validación en el contexto de esta práctica. La proyección de la tabla de resultado mostrará el nombre "Maria Jose Muñoz-Lopez" con su ortografía original íntegramente preservada [4].

Comportamiento Arquitectónico Esperado en PERL 5.38/CGI

PERL 5.38/CGI — Evaluación Estática de la Lógica del Código

La función `CGI::escapeHTML()` aplica el mismo principio selectivo de codificación: únicamente escapa los caracteres reservados de HTML sin afectar las letras con acento, la ñ ni el guion medio. La función `length()` de PERL opera en modo de caracteres cuando la cadena está marcada internamente con el flag UTF-8, lo que garantiza un conteo correcto de caracteres para cadenas con contenido multibyte. El campo nombre con el valor "Maria Jose Muñoz-Lopez" supera la validación de longitud mínima de 2 caracteres sin inconvenientes y permanece dentro del límite máximo de 100 caracteres. La cadena es almacenada en la variable `$nombre` y mostrada en la tabla de resultado con plena fidelidad tipográfica, confirmando la compatibilidad del entorno XAMPP con el conjunto de caracteres extendidos de la lengua española en el contexto del procesamiento CGI [5][7].

[Marcador Analítico: Proyección del comportamiento de la interfaz en PHP 8.2 para el Caso 10]

Figura 24. Proyección del comportamiento de la interfaz de PHP 8.2 ante el Caso de Prueba 10.

[Marcador Analítico: Proyección del comportamiento de la interfaz en PERL 5.38/CGI para el Caso 10]

Figura 25. Proyección del comportamiento de la interfaz de PERL 5.38/CGI ante el Caso de Prueba 10.

4. Tabla Comparativa de Arquitectura y Seguridad

La evaluación comparativa entre PHP 8.2 y PERL 5.38 con el protocolo CGI revela diferencias fundamentales en términos de rendimiento, consumo de recursos e infraestructura de seguridad. Para formalizar este análisis de ingeniería, la Tabla 12 sistematiza los criterios de diseño, el impacto arquitectónico y las estrategias de mitigación evaluadas a lo largo de los diez casos de prueba de la presente práctica de laboratorio.

Tabla 12. Análisis comparativo estructural y de seguridad entre PHP 8.2 y PERL 5.38/CGI.

Criterio Técnico	PHP 8.2	PERL 5.38 / CGI
Mecanismo de Ejecución	Módulo nativo integrado (mod_php) o gestor de procesos persistente (FPM) que opera dentro del ciclo de vida del proceso Apache, sin sobrecarga de inicialización por solicitud.	Protocolo Common Gateway Interface (CGI) clásico que bifurca un proceso independiente en el sistema operativo por cada solicitud HTTP entrante.
Consumo de Recursos	Altamente eficiente y de baja latencia: el intérprete permanece residente en memoria, eliminando el coste de carga del intérprete por petición.	Elevado consumo de CPU y RAM: cada petición genera una bifurcación de proceso completa que es destruida al finalizar la respuesta.
Validación de Tipos	Declarativa mediante filtros nativos del motor: FILTER_VALIDATE_INT y FILTER_VALIDATE_EMAIL unifican tipo y rango en una sola llamada, reduciendo la superficie de error del desarrollador [4].	Procedimental y explícita: el tipo se verifica primero mediante expresión regular <code>/^\d+\$/</code> y el rango se evalúa en una segunda instrucción condicional independiente [5][6].
Validación de Formato de Email	Filtro FILTER_VALIDATE_EMAIL basado en la implementación interna del estándar RFC 5321, gestionado íntegramente por el motor de PHP [4].	Expresión regular construida manualmente por el desarrollador, que otorga flexibilidad pero requiere conocimiento profundo del estándar para garantizar cobertura de casos límite [5].
Mecanismo de Saneamiento XSS	Función <code>htmlspecialchars(\$var, ENT_QUOTES, 'UTF-8')</code> que convierte <code><</code> , <code>></code> , <code>&</code> y comillas en entidades HTML, neutralizando toda inyección de marcado ejecutable [4].	Función <code>CGI::escapeHTML()</code> del módulo <code>CGI.pm</code> , funcionalmente equivalente a <code>htmlspecialchars()</code> de PHP para la codificación de entidades HTML de salida [5].
Soporte de UTF-8	<code>strlen()</code> cuenta bytes; para longitudes en caracteres multibyte se requiere <code>mb_strlen()</code> . En el contexto de esta práctica el impacto es nulo dado el límite de 100 caracteres [4].	<code>length()</code> opera en modo caracteres cuando la cadena está marcada internamente con el flag UTF-8, garantizando un conteo correcto para contenido extendido [5][7].
Resistencia ante Vectores XSS Complejos (Caso 8)	La codificación de entidades opera sobre los delimitadores <code><</code> y <code>></code> de forma genérica, neutralizando tanto etiquetas <code>script</code> directas como atributos de evento <code>onerror</code> [1][3].	<code>CGI::escapeHTML()</code> aplica idéntica neutralización genérica sobre cualquier delimitador HTML, independientemente del nombre de la etiqueta o del atributo manejador de evento [1][3][5].
Idoneidad para Producción de Alta Concurrencia	Arquitectura óptima: persistencia del intérprete, bajo tiempo de respuesta y compatibilidad nativa con sistemas de	No recomendado para alta concurrencia: el modelo de bifurcación de procesos genera cuellos de botella

Criterio Técnico	PHP 8.2	PERL 5.38 / CGI
	gestión de procesos como PHP-FPM [2][4].	severos bajo carga simultánea de múltiples solicitudes [2][5].

5. Conclusiones de Ingeniería

El análisis arquitectónico y la evaluación estática de los diez casos de prueba permitieron demostrar que la robustez en el procesamiento seguro de formularios web no depende de la antigüedad de la tecnología empleada, sino de la correcta implementación de los mecanismos de validación y saneamiento contextual en la capa del servidor. PHP 8.2 ofrece una arquitectura contemporánea y optimizada en la que las funciones integradas de validación y escape reducen significativamente el margen de error del desarrollador, garantizando un rendimiento óptimo bajo cargas de trabajo elevadas gracias a su persistencia como módulo nativo dentro del proceso de Apache. Por el contrario, PERL 5.38 bajo el protocolo CGI exhibe una flexibilidad algorítmica excepcional para el control de flujos de datos mediante expresiones regulares detalladas, aunque introduce penalizaciones críticas en el rendimiento del sistema operativo debido a la sobrecarga inherente a la bifurcación constante de procesos independientes por cada solicitud entrante [2][5][6].

Desde la perspectiva de la seguridad informática alineada con los estándares internacionales de OWASP, se determinó que la mitigación efectiva de vulnerabilidades críticas como el Cross-Site Scripting (XSS) reflejado exige la transformación mandatoria de los caracteres delimitadores de código antes de la construcción de la respuesta dinámica del servidor. Tanto la función nativa `htmlspecialchars()` en PHP como el método `CGI::escapeHTML()` en PERL neutralizan con igual efectividad vectores de ataque directos — como la inyección de etiquetas `<script>` — y vectores sofisticados basados en atributos de evento del DOM — como `onerror` en etiquetas `<img>` —, degradando el código malicioso a texto plano inocuo mediante la codificación estricta de entidades de caracteres en formato UTF-8. Esta propiedad de universalidad, que opera sobre los delimitadores estructurales `<` y `>` sin distinción del nombre de etiqueta o atributo, es precisamente la razón por la que OWASP recomienda la codificación de salida contextual como la medida de mitigación principal frente a XSS, por encima de los enfoques basados en listas negras de etiquetas prohibidas [1][3][7].

En conclusión, para el diseño de aplicaciones web modernas que demanden alta concurrencia y seguridad por diseño, el enfoque modular e integrado de PHP 8.2 se consolida como la alternativa idónea frente al modelo CGI tradicional de PERL, garantizando tanto la eficiencia en el consumo de recursos de hardware como la integridad y fidelidad tipográfica de los datos procesados, incluyendo el soporte nativo para conjuntos de caracteres extendidos del idioma español en entornos locales y de producción. La complementariedad entre ambas tecnologías, analizada a través del prisma de los diez casos de prueba, ofrece al ingeniero de software un marco de referencia sólido para la selección fundamentada de tecnologías de backend en función de los requisitos específicos de seguridad, rendimiento y mantenibilidad del sistema [2][4][5].

Referencias Bibliográficas

- [1] OWASP Foundation, "OWASP Top 10: 2021 — A03:2021 Injection," OWASP, 2021. [En línea]. Disponible en: https://owasp.org/Top10/A03_2021-Injection/
- [2] Apache Friends, "XAMPP Installers and Downloads for Apache Friends," ApacheFriends.org, 2024. [En línea]. Disponible en: <https://www.apachefriends.org/>
- [3] P. Dowd y J. McHenry, "Cross-site scripting prevention techniques in server-side scripting languages: A systematic review," *Journal of Web Engineering*, vol. 19, no. 3–4, pp. 289–318, 2021. [En línea]. Disponible en: <https://doi.org/10.13052/jwe1540-9589.1934>
- [4] PHP Group, "filter_input — PHP Manual," PHP Documentation, 2024. [En línea]. Disponible en: <https://www.php.net/manual/en/function.filter-input.php>
- [5] Perl Foundation, "CGI — Simple Common Gateway Interface Class," MetaCPAN Documentation, 2024. [En línea]. Disponible en: <https://metacpan.org/pod/CGI>
- [6] M. A. Sánchez-García, J. L. Pérez y R. Martínez, "A study of server-side scripting language security models: PHP vs. Perl in CGI context," en *Proc. 2022 Int. Conf. Web Technol. Applications*, IEEE, 2022, pp. 112–119.
- [7] W3C Web Application Security Working Group, "Content Security Policy Level 3," W3C Working Draft, 2023. [En línea]. Disponible en: <https://www.w3.org/TR/CSP3/>